SERVICE - ORIENTED SOFTWARE ENGINEERING

DT0203

Building a Python REST API to Query RDF Datasets using SPARQL

Prof. Marco Autili
University of L'Aquila
marco.autili@univaq.it

Content

- REST API Overview
- From SPARQL to REST
- Querying RDF with Python
- Designing REST Endpoints

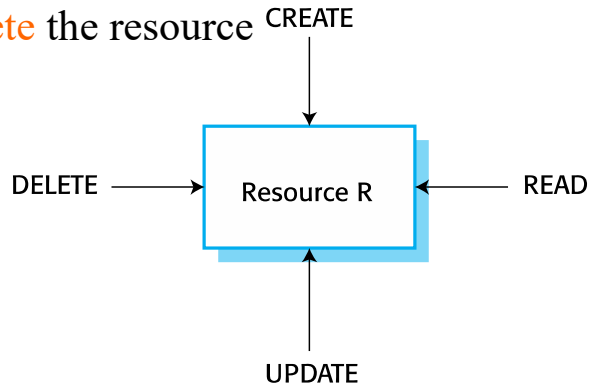
REST API Overview

- REST (Representational State Transfer)
- REST API is a way to expose data as resources over HTTP.
- A REST API is a way to access data using simple web URLs
- Each resource is identified by a URL, and we interact with it using standard methods like GET, POST, PUT, and DELETE.
- In our case, we use REST to access RDF data through simple endpoints instead of writing SPARQL queries.

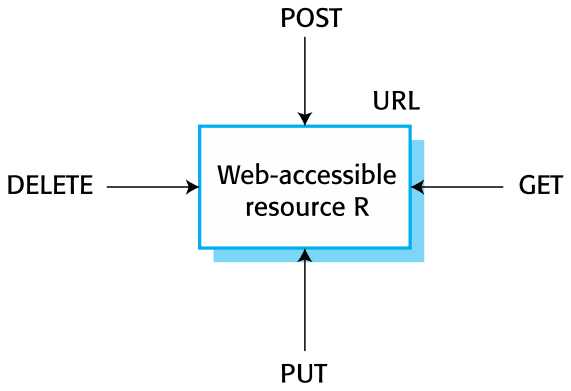
Resources and actions

The REST style underlies the web as a whole

- POST is used to create a resource. It has associated data that defines the resource
- GET is used to read the value of a resource and return that to the requestor in the specified representation, such as XHTML, that can be rendered in a web browser
- PUT is used to update the value of a resource
- DELETE is used to delete the resource



(a) General resource actions



(b) Web resources

Resource access

When a RESTful approach is used, the data is exposed and is accessed using its URI

HTTP METHODS	Collection resource, such as `https://api.example.com/collection`	Member resource, such as `https://api.example.com/collection/item3`
GET	*Retrieve* the URIs of the member resources of the collection resource in the response body.	*Retrieve* representation of the member resource in the response body.
POST	*Create* a member resource in the collection resource using the instructions in the request body. The URI of the created member resource is *automatically assigned* and returned in the response *Location* header field.	*Create* a member resource in the member resource using the instructions in the request body. The URI of the created member resource is *automatically assigned* and returned in the response *Location* header field.
PUT	*Replace* all the representations of the member resources of the collection resource with the representation in the request body, or *create* the collection resource if it does not exist.	*Replace* all the representations of the member resource or *create* the member resource if it does not exist, with the representation in the request body.
DELETE	*Delete* all the representations of the member resources of the collection resource.	*Delete* all the representations of the member resource.

https://en.wikipedia.org/wiki/Representational_state_transfer

From SPARQL to REST

- SPARQL:
  - Flexible query language for RDF
  - Requires knowledge of syntax
  - Not user-friendly for general applications

- REST API:
  - Simplifies access via predefined endpoints

Backend Process

User Request (REST) → Python API →

SPARQL Query → RDF Dataset →

JSON Response

On the backend, each REST request is translated into a SPARQL query, executed on the RDF dataset, and the result is returned in JSON format.

Example: Querying a Movie by Title

SPARQL Query

```
SELECT ?movie ?title ?director ?year ?genre
WHERE {
  ?movie dbp:originalTitle "Toy Story" .
  ?movie dbp:originalTitle ?title .
  OPTIONAL { ?movie dbp:director ?director
. }
  OPTIONAL { ?movie dbp:releaseDate ?year .
}
  OPTIONAL { ?movie dbp:genre ?genre . }
}
```

Powerful, but requires Knowledge of **SPARQL**

Translation

REST Endpoint

```
GET /movie/title/Toy%20Story
```

Simple **URL**, easy to use from a browser or applications.

Returns movie information as JSON

JSON Result


```
{
"title": "Toy Story",
"director": "John Lasseter",
"year": "1995",
"genre": [
"Adventure",
"Animation",
"Children",
"Comedy",
"Fantasy"
]
}
```


- The API returns the results in JSON format.
- This makes the data easy to read and easy to use in applications like web interfaces or other services.

HOW can we build this in Python?

Implementation Environment

- Python 3.x installed

- Required libraries:
- FastAPI
- Uvicorn
- rdflib

- RDF dataset (movies.rdf, movies-origin.rdf)

- Virtual environment (optional)

Querying RDF with Python

- RDF data loaded using rdflib
- SPARQL queries executed in Python
- FastAPI used to expose endpoints
- Results returned in JSON format

RDF Dataset → rdflib → SPARQL Query → FastAPI Endpoint → JSON Response

Implementation Example

```python
from fastapi import FastAPI
from rdflib import Graph

app = FastAPI()

g = Graph()
g.parse("movies.rdf")
```

- To develop the REST API, we first create a FastAPI application.
- Then we load the RDF dataset into a graph using rdflib.
- After that, we define REST endpoints that receive user input from the URL

```python
@app.get("/movie/title/{title}")
def get_movie_by_title(title: str):
    query = f"""
    SELECT ?movie WHERE {{
        ?movie <http://dbpedia.org/ontology/originalTitle> "{title}" .
    }}
    """
    results = g.query(query)
    return [str(row[0]) for row in results]
```

- This endpoint allows the user to search for a movie by title.
- When the user calls this URL, FastAPI captures the title parameter, Python creates a SPARQL query, rdflib executes it on the RDF dataset, and the result is returned as JSON.

Implementation Example -- Detailed

```
from fastapi import FastAPI
from rdflib import Graph
```

Here we import FastAPI to build the web service, and rdflib to work with RDF data.

- FastAPI is a Python framework that allows us to easily create REST APIs and define endpoints.
- rdflib is a Python library used to load RDF data and execute SPARQL queries on it.
- Graph is the object where RDF triples are loaded.

```
app = FastAPI()
```

We create the REST API application using FastAPI.

- app is the main object that FastAPI uses to define endpoints.

Implementation Example -- Detailed

```
g = Graph()
g.parse("movies.rdf")
```

We create an RDF graph and load the dataset from the file movies.rdf.

- RDF graph will store the triples from the dataset.
- The RDF dataset is loaded from the file movies.rdf. Once loaded, it can be queried using SPARQL

Implementation Example -- Detailed

```
@app.get("/movie/title/{title}")
```

This defines a REST endpoint where the user can provide a movie title in the URL.

- {title} is a variable part of the URL.
- This endpoint enables users to search for movies by title.

```
def get_movie_by_title(title: str):
```

This function is executed when the endpoint is called, and it receives the title as input.

- title: str means the title is expected to be text.
- When the endpoint is called, FastAPI executes this function and passes the title from the URL into it.

Implementation Example -- Detailed

```
query = f"""
SELECT ?movie WHERE {{
  ?movie <http://dbpedia.org/ontology/originalTitle> "{title}" .
}}
"""
```

We dynamically create a SPARQL query using the title provided by the user.

It searches for a movie whose originalTitle is equal to the title provided by the user.

The user-friendly REST request is translated into a SPARQL query. The title from the URL is inserted into the query.

```
results = g.query(query)
```

The query is executed on the RDF graph.

- rdflib executes the SPARQL query over the RDF graph and retrieves the matching movie resources.

```
return [str(row[0]) for row in results]
```

The results are returned as a list of movie identifiers.

Result

JSON

```
[
  "http://odws.univaq.it/movies/1"
]
```

- The result is a list containing the URI of the movie.
- It represents the movie ‘Toy Story’ inside the dataset.
- In RDF, each resource is identified by a unique URI.
- So instead of returning the full information, the query returns the identifier of the matching movie.

Result

JSON

```
[
  "http://od
]
```

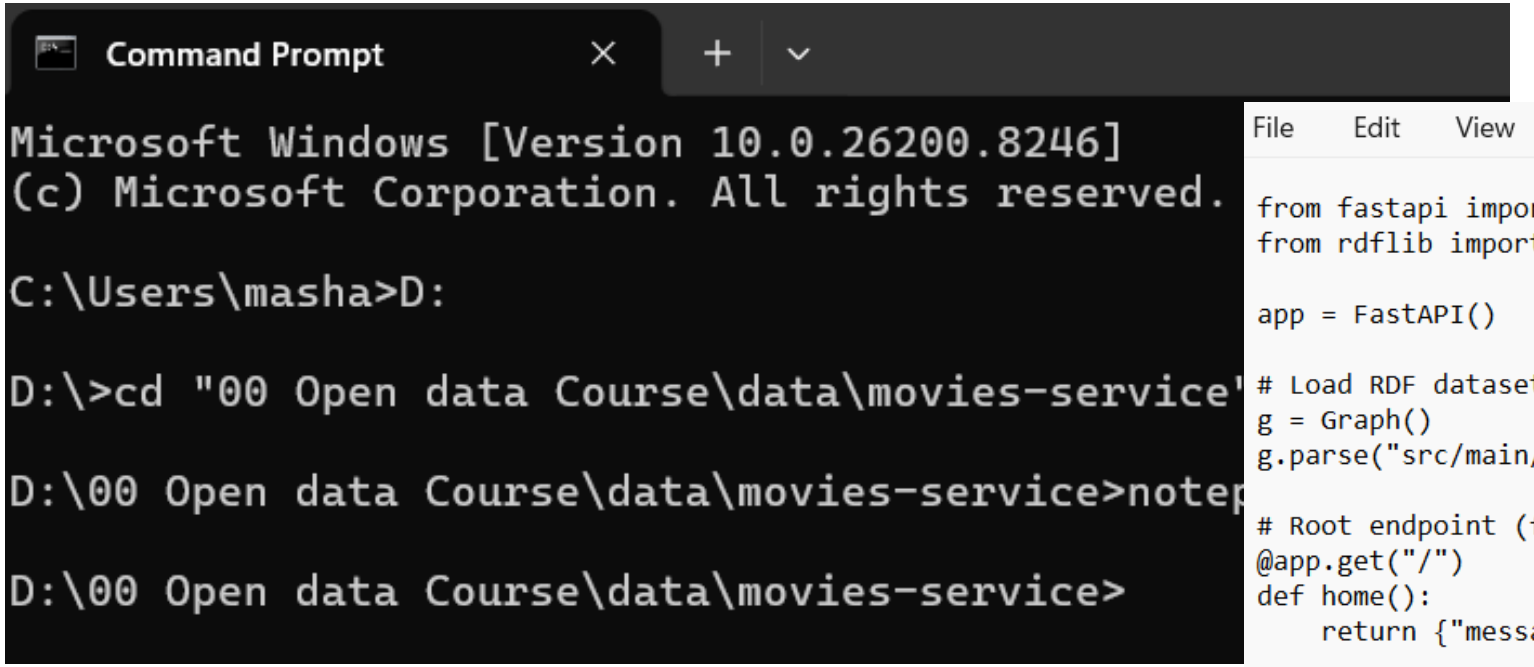
How to get the result?

URI of the movie.

movie ‘Toy Story’ inside the dataset.

- In RDF, each resource is identified by a unique URI.
- So instead of returning the full information, the query returns the identifier of the matching movie.

Example 1: Query by title – Creating app.py



File Edit View

```
from fastapi import FastAPI
from rdflib import Graph

app = FastAPI()

# Load RDF dataset
g = Graph()
g.parse("src/main/resources/movies.rdf")

# Root endpoint (for testing)
@app.get("/")
def home():
    return {"message": "Movies API is running!}

# Query movie by title
@app.get("/movie/title/{title}")
def get_movie_by_title(title: str):
    query = f"""
    SELECT ?movie WHERE {{
        ?movie <http://dbpedia.org/ontology/originalTitle> "{title}" .
    }}
    """
    results = g.query(query)
    return [str(row[0]) for row in results]
```

Example 1: Query by title – Run the API

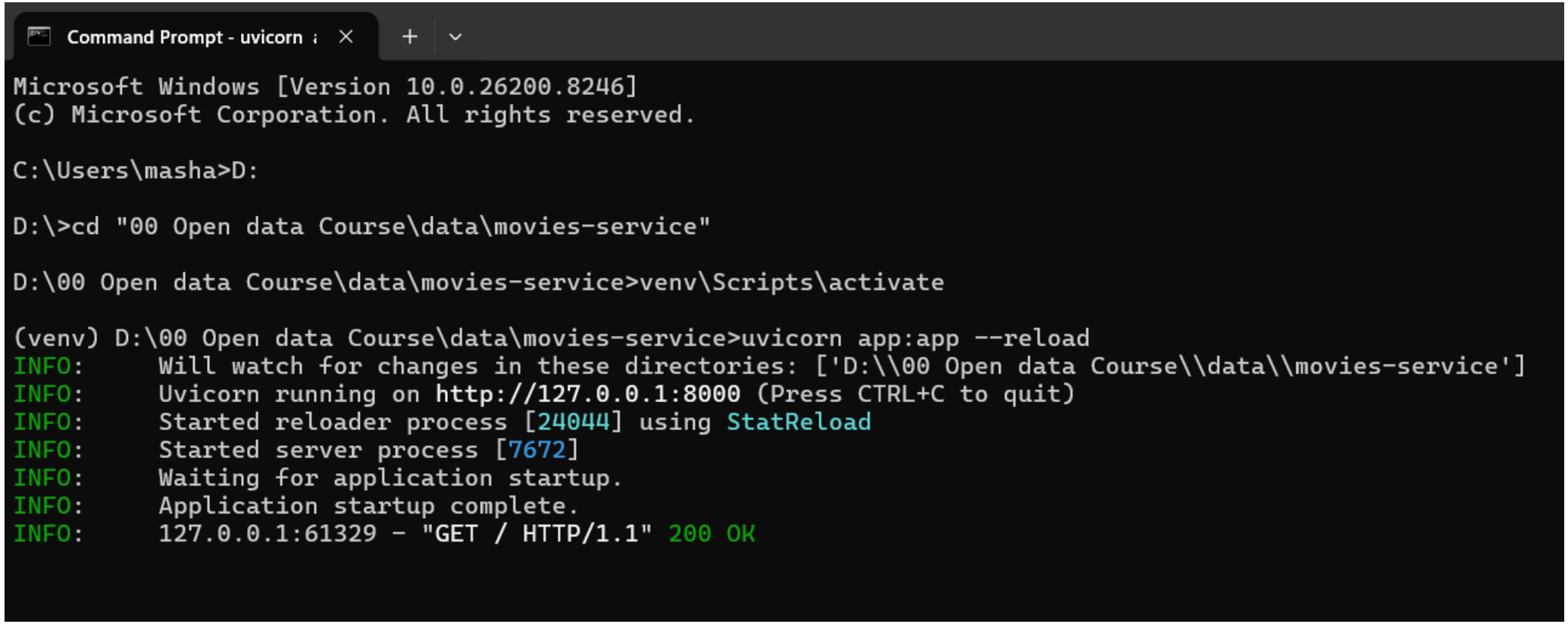
In my case, the virtual environment was already created.
If starting from scratch, we first create it using Python, then activate it and install the required libraries.
To do that, move to the dedicated folder where you have the RDF graph (your dataset).

```
python -m venv venv
venv\Scripts\activate
pip install fastapi uvicorn rdflib
uvicorn app:app --reload
```

Create your .py file using: notepad app.py

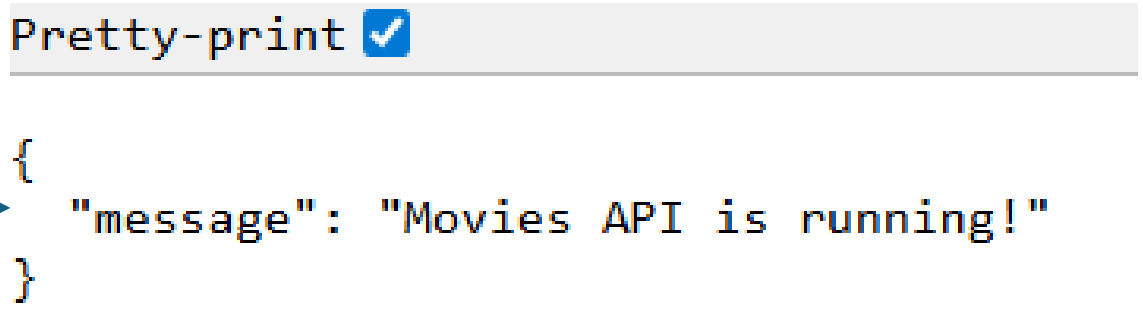
Name of your file

Example 1: Query by title – Run the API

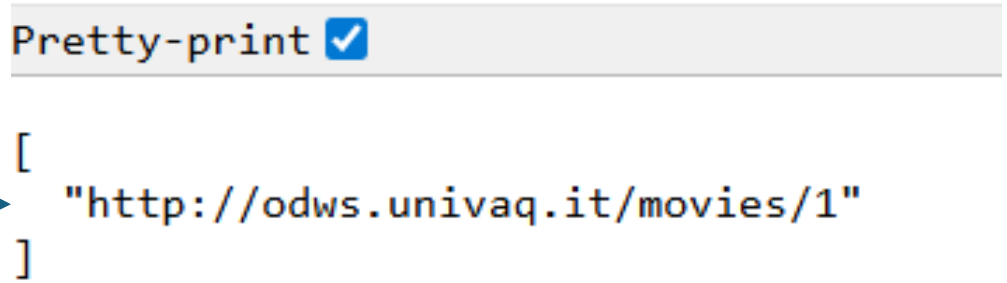


Example 1: Query by title – Results

http://127.0.0.1:8000/



http://127.0.0.1:8000/movie/title/Toy%20Story



Example 1: Query by title – Results

How to turn OFF the API
CTRL + C

http

unning!"

http://

```
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [7672]
INFO:     Stopping reloader process [24044]

(venv) D:\00 Open data Course\data\movies-service>
```

Combine multiple Datasets

Example 2: Returns FULL movie information (not just the URI)

The API returns detailed information about a movie, such as title, director, year, genre, and external links.

```python
from fastapi import FastAPI
from rdflib import Graph

app = FastAPI()

g = Graph()
g.parse("src/main/resources/movies.rdf")
g.parse("src/main/resources/movies-origin.rdf")

@app.get("/movie/title/{title}")
def get_movie_by_title(title: str):
    query = f"""
    PREFIX dbp: <http://dbpedia.org/ontology/>
    PREFIX sch: <https://schema.org/>

    SELECT ?movie ?title ?director ?year ?genre ?imdb ?wikidata
    WHERE {{
        ?movie dbp:originalTitle "{title}" .
        ?movie dbp:originalTitle ?title .
        OPTIONAL {{ ?movie dbp:director ?director . }}
        OPTIONAL {{ ?movie dbp:releaseDate ?year . }}
        OPTIONAL {{ ?movie dbp:genre ?genre . }}
        OPTIONAL {{ ?movie dbp:imdbId ?imdb . }}
        OPTIONAL {{ ?movie sch:sameAs ?wikidata . }}
    }}
    """
```

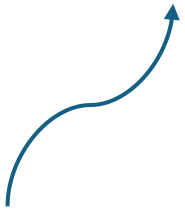
Example 2: Returns FULL movie information (not just the URI)

```
27
28      results = g.query(query)
29
30      return [
31          {
32              "movie": str(row.movie),
33              "title": str(row.title),
34              "director": str(row.director) if row.director else None,
35              "year": str(row.year) if row.year else None,
36              "genre": str(row.genre).split("|") if row.genre else None,
37              "imdb": str(row.imdb) if row.imdb else None,
38              "wikidata": str(row.wikidata) if row.wikidata else None,
39          }
40          for row in results
41      ]
```

Result 2: Returns FULL movie information (not just the URI)

```
(venv) D:\00 Open data Course\data\movies-service>uvicorn app1:app --reload
INFO:     Will watch for changes in these directories: ['D:\\00 Open data Course\\data\\movies-service']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [12148] using StatReload
INFO:     Started server process [8180]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:55757 - "GET /movie/title/Toy%20Story HTTP/1.1" 200 OK
```

http://127.0.0.1:8000/movie/title/Toy%20Story



TRY NOW: Change the movie name to see details of other movies

Pretty-print ☑


```
[
  {
    "movie": "http://odws.univaq.it/movies/1",
    "title": "Toy Story",
    "director": "John Lasseter",
    "year": "1995",
    "genre": [
      "Adventure",
      "Animation",
      "Children",
      "Comedy",
      "Fantasy"
    ],
    "imdb": "https://www.imdb.com/title/tt0114709",
    "wikidata": "https://www.wikidata.org/wiki/Q171048"
  },
  {
    "movie": "http://odws.univaq.it/movies-origin/1",
    "title": "Toy Story",
    "director": null,
    "year": "1995",
    "genre": null,
    "imdb": null,
    "wikidata": null
  }
]
```

Example 3: Find all movies released in 1995

```python
from fastapi import FastAPI
from rdflib import Graph

app = FastAPI()

g = Graph()
g.parse("src/main/resources/movies.rdf")
g.parse("src/main/resources/movies-origin.rdf")


@app.get("/")
def home():
    return {"message": "Movies API is running 🚀"}


@app.get("/movie/year/{year}")
def get_movie_by_year(year: str):
    query = f"""
    PREFIX dbp: <http://dbpedia.org/ontology/>
    PREFIX sch: <https://schema.org/>

    SELECT ?movie ?title ?director ?year ?genre ?imdb ?wikidata
    WHERE {{
        ?movie dbp:releaseDate "{year}" .
        ?movie dbp:originalTitle ?title .
        OPTIONAL {{ ?movie dbp:director ?director . }}
        OPTIONAL {{ ?movie dbp:genre ?genre . }}
        OPTIONAL {{ ?movie dbp:imdbId ?imdb . }}
        OPTIONAL {{ ?movie sch:sameAs ?wikidata . }}
    }}
    """
    results = g.query(query)

    return format_results(results)


def format_results(results):
    return [
        {
            "movie": str(row.movie),
            "title": str(row.title) if row.title else None,
            "director": str(row.director) if row.director else None,
            "year": str(row.year) if row.year else None,
            "genre": str(row.genre).split("|") if row.genre else None,
            "imdb": str(row.imdb) if row.imdb else None,
            "wikidata": str(row.wikidata) if row.wikidata else None,
        }
        for row in results
    ]
```

Example 3: Result

Copy in browser: http://127.0.0.1:8000/movie/year/1995

TRY NOW: Change the year to see more results

Pretty-print ☑


```
[
  {
    "movie": "http://odws.univaq.it/movies/1",
    "title": "Toy Story",
    "director": "John Lasseter",
    "year": null,
    "genre": [
      "Adventure",
      "Animation",
      "Children",
      "Comedy",
      "Fantasy"
    ],
    "imdb": "https://www.imdb.com/title/tt0114709",
    "wikidata": "https://www.wikidata.org/wiki/Q171048"
  },
  {
    "movie": "http://odws.univaq.it/movies/2",
    "title": "Jumanji",
    "director": "Joe Johnston",
    "year": null,
    "genre": [
      "Adventure",
      "Children",
      "Fantasy"
    ],
    "imdb": "https://www.imdb.com/title/tt0113497",
    "wikidata": "https://www.wikidata.org/wiki/Q222939"
  },
  {
    "movie": "http://odws.univaq.it/movies/3",
```

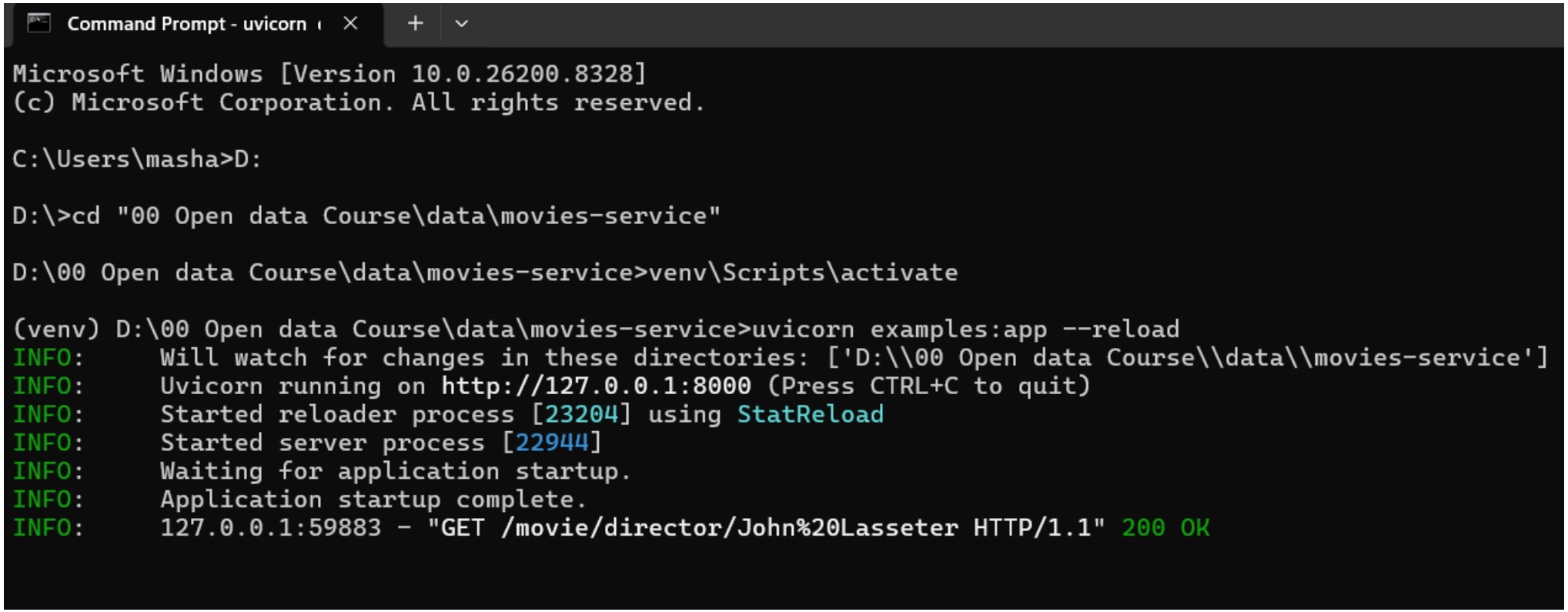
.....

```
  {
    "movie": "http://odws.univaq.it/movies/5",
    "title": "Father of the Bride Part II",
    "director": "Charles Shyer",
    "year": null,
    "genre": [
      "Comedy"
    ],
    "imdb": "https://www.imdb.com/title/tt0113041",
    "wikidata": "https://www.wikidata.org/wiki/Q1304560"
  },
  {
    "movie": "http://odws.univaq.it/movies/6",
    "title": "Heat",
    "director": "Michael Mann",
    "year": null,
    "genre": [
      "Action",
      "Crime",
      "Thriller"
    ],
    "imdb": "https://www.imdb.com/title/tt0113277",
    "wikidata": "https://www.wikidata.org/wiki/Q42198"
  },
```

Example 3: Find all movies directed by John Lasseter

```python
from fastapi import FastAPI
from rdflib import Graph

app = FastAPI()

# Load RDF datasets
g = Graph()
g.parse("src/main/resources/movies.rdf")
g.parse("src/main/resources/movies-origin.rdf")


@app.get("/")
def home():
    return {"message": "Movies API is running 🚀"}


def format_results(results):
    return [
        {
            "movie": str(row.movie),
            "title": str(row.title) if row.title else None,
            "director": str(row.director) if row.director else None,
            "year": str(row.year) if row.year else None,
            "genre": str(row.genre).split("|") if row.genre else None,
            "imdb": str(row.imdb) if row.imdb else None,
            "wikidata": str(row.wikidata) if row.wikidata else None,
        }
        for row in results
    ]

@app.get("/movie/director/{director}")
def get_movie_by_director(director: str):
    query = f"""
    PREFIX dbp: <http://dbpedia.org/ontology/>
    PREFIX sch: <https://schema.org/>

    SELECT ?movie ?title ?director ?year ?genre ?imdb ?wikidata
    WHERE {{
        ?movie dbp:director ?director .
        FILTER(CONTAINS(LCASE(STR(?director)), LCASE("{director}")))

        ?movie dbp:originalTitle ?title .
        OPTIONAL {{ ?movie dbp:releaseDate ?year . }}
        OPTIONAL {{ ?movie dbp:genre ?genre . }}
        OPTIONAL {{ ?movie dbp:imdbId ?imdb . }}
        OPTIONAL {{ ?movie sch:sameAs ?wikidata . }}
    }}
    """
    return format_results(g.query(query))
```

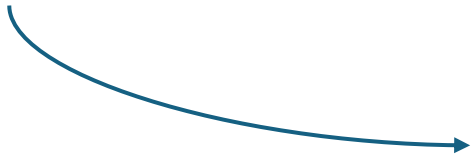
Example 3: Find all movies directed by John Lasseter



Example 3: Result

Paste in browser

http://127.0.0.1:8000/movie/director/John%20Lasseter



Pretty-print ☑

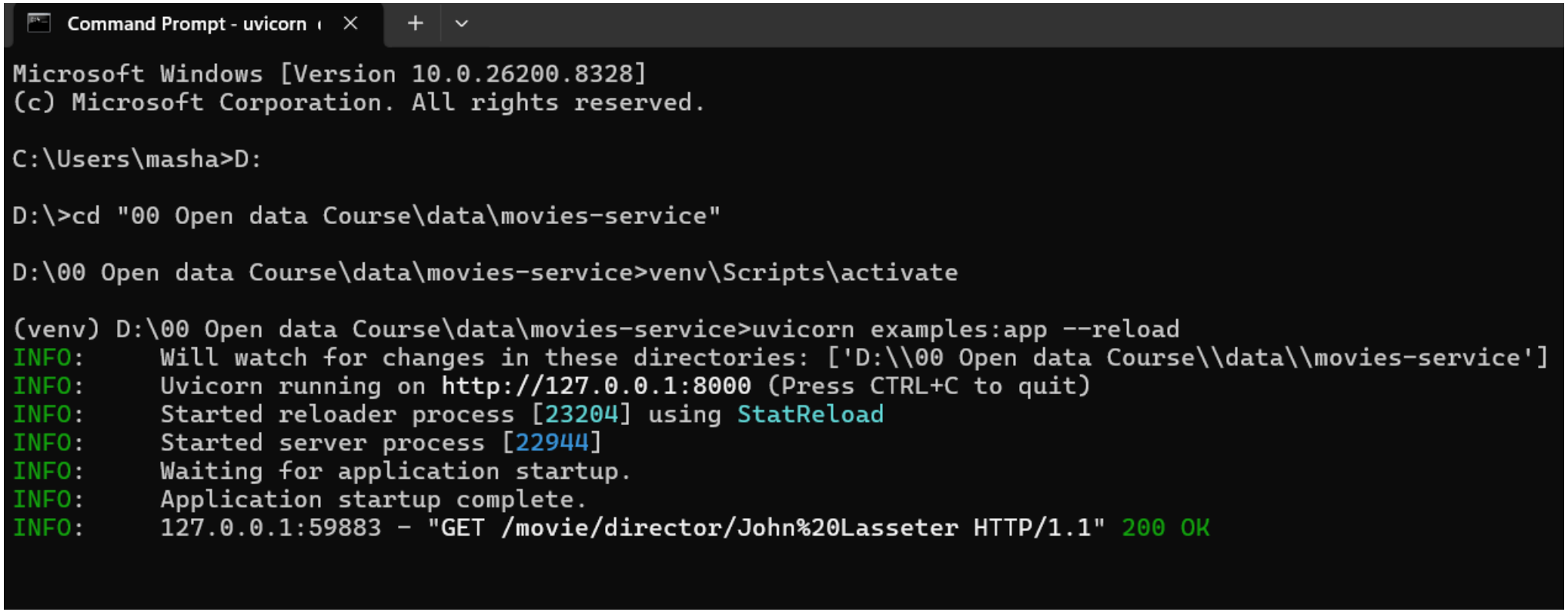

```
[
  {
    "movie": "http://odws.univaq.it/movies/1",
    "title": "Toy Story",
    "director": "John Lasseter",
    "year": "1995",
    "genre": [
      "Adventure",
      "Animation",
      "Children",
      "Comedy",
      "Fantasy"
    ],
    "imdb": "https://www.imdb.com/title/tt0114709",
    "wikidata": "https://www.wikidata.org/wiki/Q171048"
  }
]
```

Example 3: Find all movies by genre

```python
from fastapi import FastAPI
from rdflib import Graph

app = FastAPI()

# Load RDF datasets
g = Graph()
g.parse("src/main/resources/movies.rdf")
g.parse("src/main/resources/movies-origin.rdf")


@app.get("/")
def home():
    return {"message": "Movies API is running 🚀"}


def format_results(results):
    return [
        {
            "movie": str(row.movie),
            "title": str(row.title) if row.title else None,
            "director": str(row.director) if row.director else None,
            "year": str(row.year) if row.year else None,
            "genre": str(row.genre).split("|") if row.genre else None,
            "imdb": str(row.imdb) if row.imdb else None,
            "wikidata": str(row.wikidata) if row.wikidata else None,
        }
        for row in results
    ]
```

```python
@app.get("/movie/genre/{genre}")
def get_movie_by_genre(genre: str):
    query = f"""
    PREFIX dbp: <http://dbpedia.org/ontology/>
    PREFIX sch: <https://schema.org/>

    SELECT ?movie ?title ?director ?year ?genre ?imdb ?wikidata
    WHERE {{
        ?movie dbp:genre ?genre .
        FILTER(CONTAINS(LCASE(STR(?genre)), LCASE("{genre}")))

        ?movie dbp:originalTitle ?title .
        OPTIONAL {{ ?movie dbp:director ?director . }}
        OPTIONAL {{ ?movie dbp:releaseDate ?year . }}
        OPTIONAL {{ ?movie dbp:imdbId ?imdb . }}
        OPTIONAL {{ ?movie sch:sameAs ?wikidata . }}
    }}
    """
    return format_results(g.query(query))
```

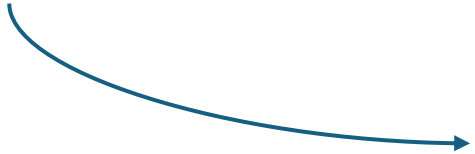
Example 3: Find all movies by genre



Example 3: Result

Paste in browser

http://127.0.0.1:8000/movie/genre/Comedy



Pretty-print ☑


```
[
  {
    "movie": "http://odws.univaq.it/movies/1",
    "title": "Toy Story",
    "director": "John Lasseter",
    "year": "1995",
    "genre": [
      "Adventure",
      "Animation",
      "Children",
      "Comedy",
      "Fantasy"
    ],
    "imdb": "https://www.imdb.com/title/tt0114709",
    "wikidata": "https://www.wikidata.org/wiki/Q171048"
  },
  {
    "movie": "http://odws.univaq.it/movies/3",
    "title": "Grumpier Old Men",
    "director": "Howard Deutch",
    "year": "1995",
    "genre": [
      "Comedy",
      "Romance"
    ],
    "imdb": "https://www.imdb.com/title/tt0113228",
    "wikidata": "https://www.wikidata.org/wiki/Q782465"
  },
  {
    "movie": "http://odws.univaq.it/movies/7",
    "title": "Sabrina",
    "director": "Sydney Pollack",
    "year": "1995",
    "genre": [
      "Comedy",
      "Romance"
    ],
    "imdb": "https://www.imdb.com/title/tt0114319",
    "wikidata": "https://www.wikidata.org/wiki/Q900708"
  },
```

Example 3: Result

Paste in browser

http://127.0.0.1:8000/movie/genre/Romance

TRY NOW: Check results for other genres

Pretty-print ☑


```
[
  {
    "movie": "http://odws.univaq.it/movies/3",
    "title": "Grumpier Old Men",
    "director": "Howard Deutch",
    "year": "1995",
    "genre": [
      "Comedy",
      "Romance"
    ],
    "imdb": "https://www.imdb.com/title/tt0113228",
    "wikidata": "https://www.wikidata.org/wiki/Q782465"
  },
  {
    "movie": "http://odws.univaq.it/movies/7",
    "title": "Sabrina",
    "director": "Sydney Pollack",
    "year": "1995",
    "genre": [
      "Comedy",
      "Romance"
    ],
    "imdb": "https://www.imdb.com/title/tt0114319",
    "wikidata": "https://www.wikidata.org/wiki/Q900708"
  },
  {
    "movie": "http://odws.univaq.it/movies/39",
    "title": "Clueless",
    "director": "Amy Heckerling",
    "year": "1995",
    "genre": [
      "Comedy",
      "Romance"
    ],
    "imdb": "https://www.imdb.com/title/tt0112697",
    "wikidata": "https://www.wikidata.org/wiki/Q931165"
  },
```

Exercise

TRY NOW

- Find movies that belong to *multiple* genres
- Find all movies that have an IMDb link available.
- Find movies that contain “toy” in the title
- Find movies released between 1990 and 2000